

Proyecto Custodia

Autorización centralizada con un Policy Enforcement Point y un Policy Decision Point (OPA/Rego) delante de un recurso financiero de referencia. Este resumen cubre el trabajo cerrado hasta ahora: Fase 0 (diseño) y Fase 1 (núcleo de autorización), validadas de punta a punta.

● Fase 0 – Diseño: completa

● Fase 1 – Autorización: completa y validada

● Fase 2 – Sensitive Data Gateway: pendiente

01 Resumen funcional

El sistema resuelve un problema concreto: nadie debería poder leer el saldo de una cuenta ajena solo porque conoce su endpoint. `custodia-pep` se para delante de `finance-api-mock` y convierte cada lectura en una decisión explícita: arma un documento de contexto (quién pregunta, qué recurso, bajo qué circunstancia) y se lo entrega a `custodia-pdp` (Open Policy Agent evaluando una política escrita en Rego). La política responde con una autorización binaria más *obligaciones* — por ejemplo, "permitido, pero enmascara el número de cuenta". El PEP nunca decide por su cuenta y nunca dejar pasar una solicitud es la respuesta por defecto ante cualquier fallo (ADR-0001, fail-closed).

Las tres reglas de negocio implementadas: un empleado puede ver su propio saldo con normalidad si su score de riesgo es bajo; con riesgo alto solo lo ve con el número de cuenta enmascarado; y un analista de fraude puede ver una cuenta ajena — enmascarada — únicamente si existe un caso abierto que lo justifique. Cualquier otra combinación se deniega por defecto.

02 Componentes desarrollados

custodia-pep

Policy Enforcement Point. Único punto de entrada a los datos de cuenta: resuelve el

custodia-pdp

Policy Decision Point. Open Policy Agent evaluando `officesec.authz`. Solo ve

sujeto, consulta el recurso, pide la decisión al PDP, aplica las obligaciones y audita.

Spring Boot 3.3.5 WebFlux :8080

sujeto/recurso/acción/contexto — nunca los datos reales de la cuenta.

OPA Rego v1 :8181

finance-api-mock

Recurso protegido. Stand-in local del proyecto real de finanzas personales; no valida quién llama — esa responsabilidad vive enteramente en el PEP.

Spring Boot 3.3.5 Web MVC :8081-18081

policies/authz.rego

3 reglas condicionales + `default deny`. Cubiertas por `authz_test.rego` : 7/7 tests en verde con `opa test`.

deny-by-default 7/7 tests

Documentación de Fase 0

`architecture.md`, `threat-model.md` (STRIDE), ADR-0001 (fail-closed vs. fail-open) y ADR-0002 (OPA/Rego vs. Amazon Verified Permissions).

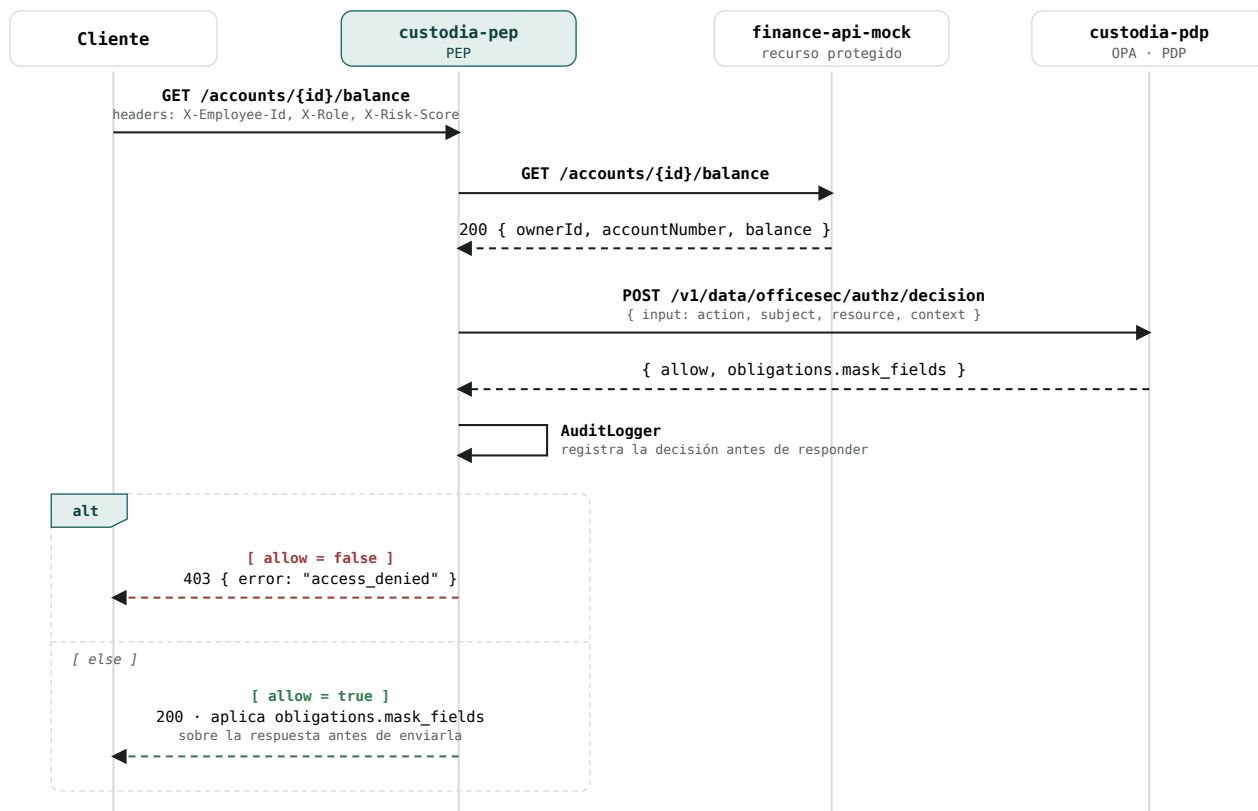
4 documentos

docker-compose.yml

Orquesta los tres servicios en una red local; monta las políticas como volumen de solo lectura dentro del PDP.

3 servicios

03 Cómo fluye una petición



El PEP nunca responde antes de tener una decisión explícita del PDP, y audita tanto lo permitido como lo denegado antes de contestar al cliente (ADR-0001, fail-closed). La flecha PEP→PDP cruza visualmente el lifeline de `finance-api-mock` porque ambos servicios son independientes entre sí — solo el PEP habla con los dos.

04 Matriz de pruebas — validada end-to-end

ESCENARIO	SUJETO (HEADERS)	RESULTADO
Dueño, riesgo bajo	emp-42 · employee · risk 10	200 — <u>datos completos</u> ✓ verificado
No es el dueño de la cuenta	emp-1 · employee · risk 5	403 — <u>access denied</u> ✓ verificado
Dueño, riesgo alto (70-90)	emp-42 · employee · risk 80	200 — <u>accountNumber enmascarado</u> ✓ verificado
Analista de fraude, sin caso abierto	emp-9 · fraud_analyst · risk 40	403 — <u>access denied</u> ✓ verificado
Analista de fraude, con caso abierto	emp-9 · fraud_analyst · risk 40 · caseOpen=true	200 — <u>accountNumber enmascarado</u> ✓ verificado

05 Bugs reales encontrados y corregidos

El valor de esta sesión no fue solo escribir el código — fue depurarlo contra un stack real (Docker, Maven, WebFlux, Jackson, OPA) hasta validarlo de punta a punta. En orden cronológico:

1 Jar sin manifest ejecutable

Al no heredar de `spring-boot-starter-parent`, se perdió el binding automático del goal `repackage`. Se agregó el `<executions>` explícito en el `spring-boot-maven-plugin`.

2 Puerto 8081 ocupado

Un contenedor propio anterior (`money-pulse-kafka-ui-1`) ya tenía el host port. Se remapeó `finance-api-mock` a `18081:8081`.

3 `docker-compose.yml` corrupto

Un pegado accidental de logs de terminal quedó incrustado en el YAML. Se reescribió el archivo completo.

4 500 — flag `-parameters` ausente

Mismo problema de raíz que el bug 1: sin `spring-boot-starter-parent` se pierde el flag de compilador que nombra los parámetros en tiempo de ejecución, y `@PathVariable String id` fallaba al resolver el nombre. Fix: `maven.compiler.parameters=true` + nombre explícito en cada `@PathVariable`.

5 500 — `argument type mismatch`

Spring WebFlux no tiene un resolver nativo para un parámetro `HttpHeaders` sin anotar en un handler — cae a un fallback que arma el argumento incorrectamente. Fix: reemplazar por `ServerWebExchange` y leer los headers con `exchange.getRequest().getHeaders()`.

6 500 — `NullPointerException` en `obligations()`

El `policy-path` en `application.yml` apuntaba al paquete (`/v1/data/officesec/authz`) en vez de a la regla (`/v1/data/officesec/authz/decision`). OPA devolvía un nivel de anidamiento extra que Jackson no podía mapear, dejando `obligations` en `null` sin lanzar ninguna excepción propia.

06 Próximos pasos

Fase 0	Diseño: arquitectura, threat model (STRIDE), ADRs. — completa.
Fase 1	Núcleo de autorización: PEP + PDP + recurso protegido. — completa y validada.
Fase 2	Sensitive Data Gateway dedicado + cifrado de campos con AWS KMS.
Fase 3	Token Broker (RFC 8693); reemplazo de los headers de desarrollo por JWT/OIDC real.
Fase 4	Casos de abuso automatizados en CI, shadow mode de políticas.
Fase 5	Observabilidad (OpenTelemetry) y runbook de incidentes.

Repositorio: github.com/diego1-avendano3/ProyectoCustodio — Fase 0-1 cerrada y verificada end-to-end con la matriz de pruebas de arriba.